

Совместное использование данных несколькими потоками

В этой главе

- Проблемы, связанные с совместным использованием данных несколькими потоками.
- Защита данных с помощью мьютексов.
- Альтернативные средства защиты совместно используемых данных.

Одним из ключевых преимуществ применения потоков для конкурентности является возможность совместного использования (разделения) данных несколькими потоками, поэтому теперь, рассмотрев запуск потоков и управление ими, обратимся к вопросам, связанным с разделением данных.

Представьте на минуту, что вы живете в одной квартире с приятелем. У вас одна кухня и одна ванная на двоих. Обычно ванной не пользуются одновременно несколько человек, и то, что сосед слишком долго плещется в воде, вынуждая вас дожидаться своей очереди, не может не раздражать. Возможно, одному из вас захочется запечь в духовке колбаски, в то время как у другого там готовятся кексы, и из этого тоже не выйдет ничего хорошего. Ну и всем знакомо чувство досады, когда при совместно используемом оборудовании вы на полпути к решению какой-нибудь задачи вдруг обнаруживаете, что кто-то взял что-то нужное вам в данный момент или что-то изменил, а вы рассчитывали, что все останется в прежнем состоянии или на своих местах.

То же самое происходит и с потоками. Если они совместно используют данные, для них нужны правила, определяющие, какой поток и к какому биту данных может получить доступ, когда и как любые обновления данных будут передаваться другим потокам, интересующимся этими данными. Но не стоит радоваться, что можно так легко разделять данные между потоками в одном процессе. Некорректное применение совместных данных — одна из основных причин ошибок, связанных с конкурентностью, и последствия могут быть куда хуже, чем кексы со вкусом колбасы.

Эта глава посвящена безопасной совместной работе с данными нескольких потоков в программах на C++, а также исключению возможных проблем и извлечению максимальных выгод.

3.1. Проблемы совместного использования данных несколькими потоками

Когда дело доходит до совместной работы с данными нескольких потоков, то все проблемы возникают из-за последствий изменения этих данных. *Если все совместно используемые данные доступны только для чтения, проблем не будет, поскольку данные, считываемые одним потоком, не зависят от того, читает другой поток те же данные или нет.* Но если один или несколько потоков, совместно использующих данные, начинают вносить в них изменения, создаются серьезные предпосылки для возникновения проблем. В таком случае следует обеспечить приемлемость конечных результатов.

Очень широкое распространение получило понятие *инвариантов*, позволяющее программистам рассуждать о коде. Это утверждения относительно конкретной структуры данных, не вызывающие сомнений, например «эта переменная содержит количество элементов в списке». В результате обновлений инварианты часто нарушаются, особенно если у структуры повышенный уровень сложности или обновление требует изменения более чем одного значения.

Рассмотрим список, имеющий две связи, в котором каждый узел содержит указатель как на следующий, так и на предыдущий узел. Один из инвариантов заключается в том, что если с узла *A* пойти по указателю «следующий» к узлу *B*, то указатель узла *B* «предыдущий» будет указывать на узел *A*. Чтобы удалить узел из списка, необходимо обновить узлы с обеих сторон, чтобы они указывали друг на друга. Как только один из узлов был обновлен, инвариант нарушается до тех пор, пока так же не будет обновлен узел на другой стороне. После завершения обновления инвариант снова соблюдается.

Этапы удаления записи из такого списка показаны на рис. 3.1.

1. Идентификация удаляемого узла — *N*.
2. Обновление ссылки в узле, предшествующем *N*, чтобы она указывала на узел, находящийся после *N*.

3. Обновление ссылки в узле после N , чтобы она указывала на узел, предшествующий узлу N .
4. Удаление узла N .

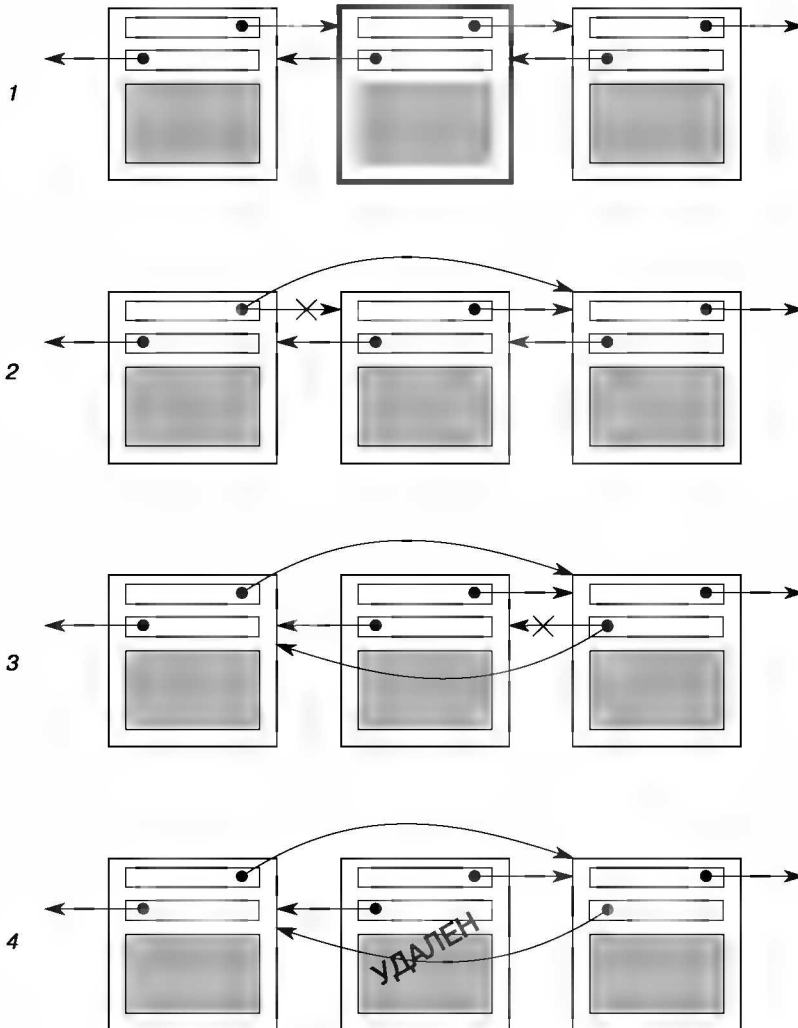


Рис. 3.1. Удаление узла из двусвязного списка

На рис. 3.1 показано, что между этапами 1 и 3 ссылки, указывающие на перемещение в прямом направлении, не согласуются со ссылками, указывающими на перемещение в обратном направлении, то есть в данном случае инвариант нарушен.

Простейшая потенциальная проблема, связанная с данными, совместно используемыми несколькими потоками, — это проблема нарушенных инвариантов.

Если не предпринять ничего, что могло бы гарантировать обратное, то в ситуации, когда один поток читает двусвязный список, а другой удаляет узел, вполне возможно, что читающий поток может увидеть список с удаленным узлом лишь частично (поскольку, как на этапе 2, была изменена только одна из ссылок), следовательно, инвариант нарушен. Последствия такого нарушения инварианта могут различаться: если на схеме другой поток читает элементы списка слева направо, то он пропустит удаляемый узел. В то же время, если второй поток попытается удалить самый правый узел на схеме, это может привести к необратимому повреждению структуры данных и в конечном итоге к сбою программы. Каким бы ни был результат, это пример одной из наиболее распространенных причин ошибок в конкурентном коде — *состояние гонки*, обуславливающее неопределенность итога.

3.1.1. Состояние гонки

Предположим, вы покупаете билет в кино. Если кинотеатр большой, билеты будут продавать сразу несколько кассиров, обслуживая одновременно несколько человек. Если кто-то в это же время покупает билет на тот же сеанс в другой кассе, то выбор места зависит от того, кто первым его закажет, вы или другой. Если осталось всего несколько мест, очередность может стать решающей: возможна настоящая гонка за последними билетами. Это пример *состояния гонки*: какое место вы получите и получите ли вообще, зависит от порядка двух покупок.

При конкурентности состоянием гонки является все, что зависит от порядка выполнения операций в двух и более потоках относительно друг друга: потоки участвуют в гонке по выполнению соответствующих операций. В большинстве случаев все заканчивается благополучно, поскольку принимаемы все возможные результаты, даже если их порядок меняется. Например, если два потока добавляют элементы в очередь обработки, то при условии сохранения инвариантов системы обычно не имеет значения, какой элемент добавляется первым. Проблема возникает, когда состояние гонки нарушает инварианты, как в приведенном ранее примере двусвязного списка. Когда речь заходит о конкурентности, выражение «состояние гонки» обычно используется для обозначения *проблемного* состояния: благополучно разрешаемое состояние гонки не вызывает интереса и не становится причиной ошибок. В стандарте C++ также определяется понятие *гонки за данными*, обозначающее конкретный тип состояния гонки, возникающий из-за одновременного изменения одного и того же объекта (подробности рассматриваются в подразделе 5.1.2). Гонки за данными вызывают опасное *неопределенное поведение*.

Критические состояния гонки обычно возникают, когда для завершения операции требуется изменение двух и более отдельных частей данных, таких как две ссылки в рассмотренном ранее примере. Поскольку операция должна получить доступ к двум отдельным частям данных, они должны быть изменены в отдельных инструкциях, и потенциально получить доступ к структуре данных, когда завершена только одна из них, может другой поток. Из-за узости окна возможностей состояние гонки зачастую трудно определить и сложно воспроизвести. Если изменения вносятся последовательными инструкциями центрального процессора, вероятность возникновения проблемы при любом проходе невелика, даже если к структуре дан-

ных в то же самое время обращается другой поток. По мере увеличения нагрузки на систему и количества выполнения операции возрастает и вероятность возникновения проблемной последовательности выполнения. Проблемы практически неизменно появляются в самое неподходящее время. Поскольку обычно состояние гонки возникает в конкретный период времени, при запуске приложения под отладчиком оно вообще может исчезать, поскольку отладчик, пусть и незначительно, но все же влияет на время выполнения программы.

При написании многопоточных программ состояние гонки может стать настоящим мучением, основные сложности создания программ, использующих конкурентность, связаны с поисками способов обхода проблемных состояний гонок.

3.1.2. Пути обхода проблемных состояний гонок

Есть несколько способов, позволяющих справиться с проблемными состояниями гонок. Самый простой вариант — заключить структуру данных в механизм защиты, чтобы гарантировать, что промежуточные состояния, в которых нарушены инварианты, будут видны только потоку, выполняющему изменения. С позиции других потоков, обращающихся к этой же структуре данных, такие изменения либо еще не начнутся, либо уже завершатся. Стандартная библиотека C++ предоставляет несколько таких механизмов, рассматриваемых в данной главе.

Есть еще один вариант — изменить конструкцию структуры данных и ее инвариантов так, чтобы модификации вносились в виде серии неделимых изменений, каждая из которых сохраняет инварианты. Обычно эта процедура называется *программированием без блокировок* (lock-free programming), и реализовать ее нелегко. В ходе работы на этом уровне возможны сложности с особенностями модели памяти и определением того, каким именно потокам какие наборы значений могут быть видны. Модель памяти рассматривается в главе 5, а программирование без блокировки — в главе 7.

Еще один способ, позволяющий справиться с состоянием гонки, заключается в обновлении структуры данных в форме *транзакции*, так же как это делается при обновлениях баз данных. Требуемая последовательность изменений и считываний данных сохраняется в журнале транзакций, а затем фиксируется в рамках единой операции. Если зафиксировать ее невозможно из-за того, что структура данных была изменена другим потоком, транзакция перезапускается. Такой способ называется *программной транзакционной памятью* (software transactional memory, STM), на момент написания данной книги в этой области велись весьма активные исследования. Непосредственная поддержка STM в языке C++ отсутствует (хотя есть техническая спецификация для транзакционных расширений памяти — Transactional Memory Extensions¹), и в книге этот способ рассматриваться не будет. Но к идее выполнения каких-либо действий в частном порядке с последующим одномоментным завершением мы еще вернемся.

¹ ISO/IEC TS 19841:2015 — Technical Specification for C++ Extensions for Transactional Memory http://www.iso.org/iso/home/store/catalogue_tc/catalogue_detail.htm?csnumber=66343.

Основным механизмом защиты совместно используемых данных, обеспеченным стандартом C++, является *мьютекс*, поэтому сначала будет рассмотрен именно он.

3.2. Защита совместно используемых данных с применением мьютексов

Итак, имеется совместно используемая структура данных, например связанный список из предыдущего раздела, и его нужно защитить от состояния гонки и возможных нарушений инвариантов. Наверное, неплохо было бы получить возможность пометчать все фрагменты кода, обращающиеся к структуре данных, как *взаимоисключающие*, чтобы при выполнении одного из них каким-либо потоком любой другой поток, пытающийся получить доступ к этой структуре данных, был бы вынужден ждать, пока первый поток не завершит выполнение такого фрагмента. Тогда поток не смог бы увидеть нарушенный инвариант, кроме тех случаев, когда он сам выполнял бы модификацию.

Данное желание не из разряда сказочных, именно это будет получено при использовании примитива синхронизации под названием «*мьютекс*», означающего *взаимное исключение* (*mutual exclusion*). Перед получением доступа к совместно используемой структуре данных мьютекс, связанный с ней, *блокируется*, а когда доступ к ней заканчивается, *блокировка с него снимается*. Библиотека потоков гарантирует, что, как только один поток заблокирует определенный мьютекс, все остальные потоки, пытающиеся его заблокировать, должны будут ждать, пока поток, который успешно заблокировал мьютекс, его не разблокирует. Тем самым гарантируется, что все потоки видят непротиворечивое представление совместно используемых данных без нарушенных инвариантов.

Мьютексы — главный механизм защиты данных, доступный в C++, но панацеей от всех бед их не назовешь: важно структурировать код таким образом, чтобы защитить нужные данные (см. подраздел 3.2.2) и избежать состояний гонки, присущих используемым интерфейсам (см. подраздел 3.2.3). У мьютексов имеются и собственные проблемы в виде *взаимной блокировки* (см. подраздел 3.2.4) и защиты либо слишком большого, либо слишком малого объема данных (см. подраздел 3.2.8).

Начнем с основ.

3.2.1. Использование мьютексов в C++

Мьютекс в C++ получается при создании экземпляра `std::mutex`, блокируется вызовом компонентной функции `lock()`, а разблокируется вызовом компонентной функции `unlock()`. Но напрямую вызывать компонентные функции не рекомендуется, поскольку при этом нужно помнить о необходимости вызова `unlock()` на всех путях кода из функции, в том числе возникающих из-за выдачи исключений. Вместо этого стандартной библиотекой C++ предоставляется онекун блокировки — шаблон класса `std::lock_guard`, который реализует для мьютекса способ RAII. Он блокиру-

ет предоставляемый мьютекс при работе конструктора и разблокирует при работе деструктора, гарантируя правильную разблокировку в любом случае. В листинге 3.1 показано, как с помощью `std::mutex`, а также `std::lock_guard` защитить список, к которому могут получить доступ сразу несколько потоков. Оба класса объявлены в заголовке `<mutex>`.

Листинг 3.1. Защита списка с помощью мьютекса

```
#include <list>
#include <mutex>
#include <algorithm>
std::list<int> some_list; ← ❶
std::mutex some_mutex; ← ❷
void add_to_list(int new_value)
{
    std::lock_guard<std::mutex> guard(some_mutex); ← ❸
    some_list.push_back(new_value);
}
bool list_contains(int value_to_find)
{
    std::lock_guard<std::mutex> guard(some_mutex); ← ❹
    return std::find(some_list.begin(), some_list.end(), value_to_find)
        != some_list.end();
}
```

В листинге 3.1 есть одна глобальная переменная ❶, она защищена соответствующим глобальным экземпляром `std::mutex` ❷. Использование `std::lock_guard<std::mutex>` в `add_to_list()` ❸ и еще раз в `list_contains()` ❹ означает, что доступ к списку из этих функций является взаимноисключающим: `list_contains()` никогда не увидит список на промежуточном этапе его изменения функцией `add_to_list()`.

В C++17 появилась новая возможность, называемая *выводом аргументов шаблона класса* (class template argument deduction), которая означает, что для простых шаблонов классов, таких как `std::lock_guard`, список аргументов шаблона зачастую можно не указывать. Строки кода ❸ и ❹ в компиляторе C++17 можно сократить до следующего вида:

```
std::lock_guard guard(some_mutex);
```

В подразделе 3.2.4 будет показано, что в C++17 также введена улучшенная версия опекуна блокировки под названием `std::scoped_lock`, поэтому в среде C++17 этот код можно записать следующим образом:

```
std::scoped_lock guard(some_mutex);
```

Для того, чтобы код был понятнее, и для совместимости с прежними компиляторами в других фрагментах кода по-прежнему будет использоваться шаблон класса `std::lock_guard` и указываться его аргументы.

Иногда в таком использовании глобальных переменных есть определенный смысл, однако в большинстве случаев мьютекс и защищенные данные помещаются в один класс, а не в глобальные переменные. Это соответствует стандартным

правилам объектно-ориентированного проектирования: помещение их в один класс служит признаком связности друг с другом, позволяя инкапсулировать функциональность и обеспечить защиту. В данном случае `add_to_list` и `list_contains` станут компонентными функциями класса, а мьютекс и защищенные данные — закрытыми компонентами класса, что значительно упростит определение того, какой код имеет доступ к данным и, следовательно, какой код должен заблокировать мьютекс. Если все компонентные функции класса блокируют мьютекс перед доступом к любым другим компонентным данным и разблокируют его по завершении доступа, данные будут надежно защищены от любого обращающегося к ним кода.

Самый проницательный читатель мог бы заметить, что это не совсем так: если одна из компонентных функций возвращает указатель или ссылку на защищенные данные, то обстоятельства блокировки мьютекса всеми компонентными функциями строго по очереди уже не будут иметь значения, потому что тем самым в защите будет проделана большая дыра. *Теперь обратиться к защищенным данным и, возможно, их изменить, не блокируя мьютекс, сможет любой код, имеющий доступ к этому указателю или ссылке.* Поэтому защита данных с помощью мьютекса требует тщательной проработки интерфейса, чтобы гарантировать, что мьютекс заблокирован до того, как появится доступ к защищенным данным, и к ним не будет никаких черных ходов.

3.2.2. Структуризация кода для защиты совместно используемых данных

Как видите, защита данных с помощью мьютекса не сводится лишь к добавлению объекта `std::lock_guard` в каждую компонентную функцию: один случайный указатель или ссылка — и вся эта защита рухнет. На одном и том же уровне проверка на наличие случайных указателей или ссылок довольно проста: данные находятся в безопасности до тех пор, пока ни одна из компонентных функций не возвращает указатель или ссылку на защищенные данные вызывающему коду посредством возвращаемого значения или выходного параметра. Если же копнуть немного глубже, то все, как всегда, становится гораздо сложнее. Помимо проверки того, что компонентные функции не передают указатели или ссылки вызывающему их коду, важно также убедиться, что они не передают эти указатели или ссылки тем функциям, которые вызываются ими и не контролируются вами. Такая передача не менее опасна: эти функции могут хранить указатель или ссылку в том месте, где их позже можно использовать без защиты, предоставляемой мьютексом. В этом смысле особенно опасны функции, которые предоставляются во время выполнения программы в виде аргументов функций или иными способами (листинг 3.2).

Листинг 3.2. Случайная передача за пределы защиты ссылки на защищаемые данные

```
class some_data
{
    int a;
```

```

    std::string b;
public:
    void do_something();
};
class data_wrapper
{
private:
    some_data data;
    std::mutex m;
public:
    template<typename Function>
    void process_data(Function func)
    {
        std::lock_guard<std::mutex> l(m);
        func(data);
    }
};
some_data* unprotected;
void malicious_function(some_data& protected_data)
{
    unprotected=&protected_data;
}
data_wrapper x;
void foo()
{
    x.process_data(malicious_function);
    unprotected->do_something();
}

```

1 Передача «защищенных» данных функции, предоставленной пользователем

 2 Передача во вредоносную функцию

 3 Незащищенный доступ к защищенным данным

В этом примере код в `process_data`, казалось бы, не вызывает опасений и выглядит вполне защищенным с помощью `std::lock_guard`, но вызов предоставленной пользователем функции `func` ❶ означает, что `foo` может передать вредоносную функцию `malicious_function` для обхода защиты ❷, а затем вызвать `do_something()` уже без блокировки мьютекса ❸.

По сути, проблема кода состоит в невыполнении им своих намерений пометить все фрагменты кода, обращающиеся к структуре данных, как *взаимноисключающие*. В данном случае он пропустил код в `foo()`, который вызывает `unprotected->do_something()`. К сожалению, помочь справиться с проблемой такого рода библиотека потоков C++ не в состоянии, так как задача блокировки чужого мьютекса для защиты данных возлагается на программиста. В то же время можно воспользоваться рекомендацией, которая поможет в подобных случаях: *не передавайте указатели и ссылки на защищенные данные за пределы блокировки никаким способом: ни возвращая их из функции, ни сохраняя во внешне видимой памяти и ни передавая в качестве аргументов функциям, предоставленным пользователем*.

Хотя данная ошибка при попытке использования мьютексов для защиты совместно используемых данных распространена наиболее широко, она далеко не единственно возможная. В следующем разделе будет показано, что даже при защите данных мьютексом вероятность возникновения состояния гонки все еще сохраняется.

3.2.3. Обнаружение состояния гонки, присущего интерфейсам

Применение мьютекса или другого механизма для защиты совместно используемых данных не даст полной гарантии защищенности от состояния гонки, нужно еще убедиться в защищенности соответствующих данных. Вернемся к примеру двусвязного списка. Чтобы поток мог безопасно удалить узел, необходимо убедиться в том, что предотвращен конкурентный доступ к трем узлам: удаляемому узлу и узлам по обе стороны от него. Если защищать доступ к указателям каждого узла по отдельности, ситуация будет не лучше, чем при использовании кода, в котором мьютексов нет, поскольку от вероятности состояния гонки избавиться не удастся — нужно защищать не отдельные узлы на отдельных этапах, а всю структуру данных для всей операции удаления. В данном случае самое простое решение — применить один мьютекс, защищающий, как в листинге 3.1, весь список.

Безопасность отдельных операций над списком не решает общей проблемы, состояние гонки может возникнуть даже при использовании простого интерфейса. Рассмотрим структуру данных стека на примере применения адаптера контейнера `std::stack`, показанного в листинге 3.3. Помимо конструкторов и функции `swap()`, есть только пять операций, проводимых над `std::stack`: можно поместить в стек новый элемент, применив функцию `push()`, извлечь элемент из стека, использовав `pop()`, прочесть верхний элемент с помощью `top()`, проверить, не является ли стек пустым, задействовав `empty()`, и прочесть количество элементов стека, применив `size()`. Даже если изменить функцию `top()`, заставив ее возвращать копию, а не ссылку (в соответствии с указаниями из подраздела 3.2.2), и защитить внутренние данные с помощью мьютекса, этот интерфейс все равно не будет застрахован от возникновения гонки. Такая проблема характерна не только для реализаций на основе мьютексов, она свойственна и интерфейсу, поэтому состояния гонки будут возникать и при реализации кода без блокировок.

Листинг 3.3. Интерфейс адаптера контейнера `std::stack`

```
template<typename T, typename Container=std::deque<T> >
class stack
{
public:
    explicit stack(const Container&);
    explicit stack(Container&& = Container());
    template <class Alloc> explicit stack(const Alloc&);
    template <class Alloc> stack(const Container&, const Alloc&);
    template <class Alloc> stack(Container&&, const Alloc&);
    template <class Alloc> stack(stack&&, const Alloc&);
    bool empty() const;
    size_t size() const;
    T& top();
    T const& top() const;
    void push(T const&);
    void push(T&&);
    void pop();
    void swap(stack&&);
    template <class... Args> void emplace(Args&&... args); ← Новое в C++14
};
```

Проблема в том, что полагаться на результаты работы функций `empty()` и `size()` нельзя. Хотя на момент вызова они, вероятно, и были достоверными, но после возврата из функции любой другой поток может обратиться к стеку и затолкнуть в него новые элементы (`push()`), либо вытолкнуть существующие (`pop()`), причем до того, как поток, вызывающий `empty()` или `size()`, сможет воспользоваться этой информацией.

В частности, если экземпляр стека *не является совместно используемым*, то с помощью следующего кода вполне безопасно проверить его на пустоту методом `empty()`, а затем вызвать `top()` для доступа к верхнему элементу:

```
stack<int> s;  
if(!s.empty()) ← ❶  
{  
    int const value=s.top(); ← ❷  
    s.pop(); ← ❸  
    do_something(value);  
}
```

Для однопоточного кода такой вариант не только безопасен, но и вполне ожидаем, поскольку вызов `top()` при пустом стеке вызывает неопределенное поведение. При совместно используемом объекте стека *эта последовательность вызовов перестает быть безопасной* из-за возможности вызова `pop()` из другого потока, в результате чего между вызовами `empty()` ❶ и `top()` ❷ удаляется последний элемент. Следовательно, это классическое состояние гонки, не устраняемое внутренним использованием мьютекса для защиты содержимого стека и являющееся следствием применения данного интерфейса.

Так каким же должно быть решение? Данная проблема является следствием конструкции интерфейса, поэтому решение состоит в ее изменении. Остается ответить на вопрос: какими должны быть изменения? Проще всего объявить, что функция `top()` выдаст исключение, если при ее вызове в стеке не будет никаких элементов. Это позволит полностью решить проблему, однако трудоемкость программирования возрастет, поскольку теперь требуется перехватывать исключения, даже если вызов `empty()` вернул значение `false`. Вызов `empty()` становится не необходимой частью конструкции, а средством оптимизации, позволяющим избежать издержек на создание исключения, если стек уже пуст (хотя, если между вызовами `empty()` и `top()` состояние изменяется, исключение все равно будет выдано).

Если присмотреться к предыдущему фрагменту кода, можно заметить вероятность возникновения еще одного состояния гонки между вызовами `top()` ❷ и `pop()` ❸. Рассмотрим два потока, выполняющие предыдущий фрагмент кода и ссылающиеся на один и тот же объект стека `s`. Это вполне типичная ситуация: при использовании потоков для повышения производительности зачастую достаточно иметь несколько потоков, выполняющих один и тот же код для разных данных, и совместно используемый объект стека идеально подходит для разделения работы между ними (хотя чаще всего для этой цели используется очередь — см. примеры в главах 6 и 7). Предположим, что изначально в стеке имеется два элемента, поэтому не нужно беспокоиться о гонке между `empty()` и `top()` в любом потоке и учитывать возможные шаблоны выполнения.

Если стек внутренне защищен мьютексом, то компонентную функцию стека в любой момент времени может запускать только один поток, поэтому вызовы таких функций пугным образом чередуются, но вызовы `do_something()` могут выполняться одновременно. Один из возможных вариантов выполнения кода показан в табл. 3.1.

Таблица 3.1. Возможный порядок операций над стеком из двух потоков

Поток А	Поток Б
<pre>if(!s.empty()) int const value=s.top(); s.pop(); do_something(value);</pre>	<pre>if(!s.empty()) int const value=s.top(); s.pop(); do_something(value);</pre>

Если это единственные запущенные потоки, то можно заметить, что между двумя вызовами `top()` никаких изменений в стек не вносится, поэтому оба потока будут видеть одно и то же значение. При этом между вызовами `pop()` нет вызовов `top()`. Следовательно, одно из двух значений в стеке отбрасывается непрочитанным, а другое обрабатывается дважды. И это еще одно состояние гонки, гораздо более коварное, чем неопределенное поведение в случае гонки между `empty()` и `top()`. В этом коде вроде бы вообще нет ничего явно неправильного, а последствия ошибки, вероятно, проявятся намного позже причины ее возникновения, хотя вполне очевидно, что они зависят от действий, выполняемых функцией `do_something()`. Здесь требуется более радикальное изменение интерфейса, объединяющего вызовы `top()` и `pop()` под защитой мьютекса. Том Каргилл (Tom Cargill)¹ отметил, что такой объединенный вызов способен вызвать проблемы, если конструктор копирования для объектов в стеке может выдать исключение. А Херб Саттер (Herb Sutter)² всесторонне рассмотрел эту проблему с точки зрения безопасности исключений, но вероятность возникновения состояния гонки привносит в нее новые аспекты.

Чтобы разъяснить суть проблемы, рассмотрим стек `<vector<int>>`. Теперь вектор является контейнером динамически изменяемого размера, поэтому при копировании вектора библиотека должна распределить из кучи достаточный объем памяти, позволяющий скопировать содержимое. При высокой загрузке системы или существенном дефиците ресурсов распределение памяти может дать сбой, а конструктор копирования вектора — выдать исключение `std::bad_alloc`. Такое развитие событий вполне вероятно, когда вектор содержит слишком много

¹ Cargill T. Exception Handling: A False Sense of Security // C++ Report 6, 1994. — № 9. Статья также доступна по адресу http://www.informit.com/content/images/020163371x/supplements/Exception_Handling_Article.html.

² Sutter H. Exceptional C++: 47 Engineering Puzzles, Programming Problems, and Solutions. — Addison Wesley Professional, 1999.

элементов. Если в определении функции `pop()` были предусмотрены возврат извлеченного значения и удаление его из стека, то мы получаем предпосылку возникновения проблемы: извлекаемое значение возвращается вызывающему коду только после изменения стека, но копирование данных для их возврата вызывающему коду может выдать исключение. Если это произойдет, извлеченные данные будут потеряны: удаление из стека уже произошло, а при копировании случился сбой! Разработчики интерфейса `std::stack` любезно разбивают операцию на две части: получение верхнего элемента (`top()`), а затем удаление его из стека (`pop()`), чтобы при невозможности безопасного копирования данных элемент оставался в стеке. Если проблема заключалась в дефиците памяти кучи, то, вероятно, приложение в состоянии освободить часть памяти и повторить попытку.

К сожалению, именно такого разбиения мы пытаемся избежать для устранения состояния гонки! К счастью, есть альтернативные варианты, но они сопряжены с определенными затратами.

Вариант 1: передача в ссылке

Первый вариант заключается в передаче функции `pop()` в виде аргумента ссылки на переменную, в которую желательно получить извлекаемое из стека значение:

```
std::vector<int> result;  
some_stack.pop(result);
```

Этот вариант подходит для многих случаев, но у него есть явный недостаток: он требует от вызывающего кода создания экземпляра типа значения стека еще до вызова, чтобы его можно было передать в качестве получателя. Для некоторых типов это неприемлемо из-за затратности создания экземпляра с точки зрения времени или ресурсов. Другим типам такая возможность предоставляется не всегда, поскольку конструкторам требуются параметры, которые могут быть доступны в данном месте кода. И наконец, нужно, чтобы сохраненный тип допускал присваивание значений. Это очень важное ограничение: многие пользовательские типы не поддерживают присваивание, хотя могут поддерживать конструкции перемещения или даже копирования и допускают возвращение результата в виде значения.

Вариант 2: востребование конструктора копирования без выдачи исключений или конструктора перемещения

Остается решить проблему безопасности при выдаче исключений в функции `pop()`, возвращающей значение, если возвращение результата в виде значения может выдать исключение. У многих типов имеются конструкторы копирования, не выдающие исключений, а с новой поддержкой ссылок на *r*-значения в стандарте C++ (см. раздел A.1) у многих других типов будут конструкторы перемещения, которые не выдают исключений, даже если они могут выдаваться их конструктором копирования. Один из приемлемых вариантов заключается в ограничении использования вашего потокобезопасного стека теми типами, которые могут безопасно возвращать результаты в виде значений без выдачи исключения.

При всей своей безопасности это решение далеко от идеала. Несмотря на возможность обнаружения в ходе компиляции конструктора копирования или перемещения, не выдающего исключений, при использовании признаков типа `std::is_nothrow_copy_constructible` и `std::is_nothrow_move_constructible` данное решение накладывает слишком большие ограничения. Пользовательских типов, имеющих конструкторы копирования, выдающие исключения, и не имеющих конструкторов перемещений, гораздо больше, чем типов с конструкторами копирования и/или перемещения, не выдающими исключений (хотя по мере того, как все привыкнут к поддержке в C++11 ссылок на r-значения, ситуация может измениться). Будет жаль, если такие типы окажется невозможно сохранить в потокобезопасном стеке.

Вариант 3: возвращение указателя на извлекаемый элемент

Третий вариант заключается не в возвращении элемента в виде значения, а в возвращении указателя на извлекаемый элемент. Его преимуществом является свободное копирование указателей без выдачи исключений, что позволяет избежать проблемы исключений, рассмотренных Каргиллом. Недостаток заключается в том, что для возвращения указателя требуются средства управления памятью, распределенной объекту, а для простых типов, например целых чисел, поддержки такого управления памятью могут превышать затраты на возвращение типа в виде значения. Для любого интерфейса, использующего данный вариант, при выборе типа для указателя вполне подойдет `std::shared_ptr`, он не только позволит избежать утечек памяти, поскольку объект уничтожается после уничтожения последнего указателя, но и даст возможность библиотеке полностью контролировать схему распределения памяти без использования `new` и `delete`. Это может сыграть важную роль в оптимизации: требование, чтобы каждый объект в стеке распределялся отдельно с помощью `new`, приводило бы к более весомым издержкам по сравнению с пеходной непотокобезопасной версией.

Вариант 4: сочетание первого и второго или третьего вариантов

Гибкость исключать не следует, особенно в обобщенном коде. При выборе варианта 2 или 3 сравнительно несложно реализовать и вариант 1, что даст пользователям кода возможность выбрать наиболее подходящий для них вариант с небольшими дополнительными затратами.

Пример определения потокобезопасного стека

В листинге 3.4 показано определение класса для стека, не подверженного состоянию гонки в интерфейсе с реализацией первого и третьего вариантов: в нем есть две перегружаемые версии функции `pop()`, одна из которых принимает ссылку на место хранения значения, а другая возвращает `std::shared_ptr<T>`. Используется довольно простой интерфейс, состоящий всего из двух функций, `push()` и `pop()`.

Листинг 3.4. Схематическое определение класса потокобезопасного стека

```

#include <exception>
#include <memory>
struct empty_stack: std::exception
{
    const char* what() const noexcept;
};
template<typename T>
class threadsafe_stack
{
public:
    threadsafe_stack();
    threadsafe_stack(const threadsafe_stack&);
    threadsafe_stack& operator=(const threadsafe_stack&) = delete;
    void push(T new_value);
    std::shared_ptr<T> pop();
    void pop(T& value);
    bool empty() const;
};

```

Для `std::shared_ptr<>`

Оператор присваивания удален

Упрощение интерфейса позволило добиться максимальной безопасности, ограничены даже операции целиком над всем стеком. Сам стек нельзя присвоить, поскольку удален оператор присваивания ❶ (см. раздел А.2) и отсутствует функция `swap()`. Но его можно скопировать, так как элементы стека разрешено копировать. Если стек пуст, функции `pop()` выдают исключение `empty_stack`, поэтому все должно работать, даже если стек был изменен после вызова функции `empty()`. Как упоминалось в описании варианта 3, применение `std::shared_ptr` позволяет стеку обеспечить распределение памяти и обойтись без лишних обращений к `new` и `delete`. Из пяти операций со стеком осталось только три: `push()`, `pop()` и `empty()`. И даже `empty()` лишняя. Такое упрощение интерфейса позволяет обеспечить более жесткий контроль над использованием данных: теперь можно гарантировать, что мьютекс блокируется для всей совокупности операций. В листинге 3.5 показана простая реализация, служащая оболочкой `std::stack<>`.

Листинг 3.5. Дополненное определение класса потокобезопасного стека

```

#include <exception>
#include <memory>
#include <mutex>
#include <stack>
struct empty_stack: std::exception
{
    const char* what() const throw();
};
template<typename T>
class threadsafe_stack
{
private:
    std::stack<T> data;
    mutable std::mutex m;
};

```

```

public:
    threadsafe_stack(){}
    threadsafe_stack(const threadsafe_stack& other)
    {
        std::lock_guard<std::mutex> lock(other.m);
        data=other.data;
    }
    threadsafe_stack& operator=(const threadsafe_stack&) = delete;
    void push(T new_value)
    {
        std::lock_guard<std::mutex> lock(m);
        data.push(std::move(new_value));
    }
    std::shared_ptr<T> pop()
    {
        std::lock_guard<std::mutex> lock(m);
        if(data.empty()) throw empty_stack();
        std::shared_ptr<T> const res(std::make_shared<T>(data.top()));
        data.pop();
        return res;
    }
    void pop(T& value)
    {
        std::lock_guard<std::mutex> lock(m);
        if(data.empty()) throw empty_stack();
        value=data.top();
        data.pop();
    }
    bool empty() const
    {
        std::lock_guard<std::mutex> lock(m);
        return data.empty();
    }
};

```

Копирование выполняется в теле конструктора

Проверка стека на пустоту перед попыткой извлечения элемента

Размещение возвращаемого значения перед изменением стека

Эта реализация стека *допускает копирование* — конструктор копирования блокирует мьютекс в исходном объекте, а затем копирует внутренний стек. Чтобы при этом обеспечить удержание мьютекса, копирование выполняется в теле конструктора ❶, а не в списке инициализаторов компонентов.

При рассмотрении функций `top()` и `pop()` становится понятно, что проблемное состояние гонки в интерфейсах возникает из-за блокировки при слишком малой глубине детализации, так как защита не охватывает всю нужную операцию. Проблемы с мьютексами возможны также из-за блокировки при слишком большой глубине детализации: чрезвычайные обстоятельства возникают в результате применения одного глобального мьютекса, защищающего все совместно используемые данные. В системе со значительным объемом совместно используемых данных это обстоятельство способно свести на нет любые преимущества повышения производительности, связанные с конкурентностью, поскольку потоки вынуждены запускаться поодиночке даже при обращении к разным битам данных. Первые версии ядра Linux, предназначенные для работы в многопроцессорных системах, использовали единую

глобальную блокировку ядра. При этом у двухпроцессорной системы производительность обычно оказывалась ниже, чем у двух однопроцессорных систем, а производительность четырехпроцессорной системы была далеко не такой, как у четырех однопроцессорных систем. Вокруг такого ядра велось множество споров, поскольку потоки, запущенные на дополнительных процессорах, не могли выполнять полезную работу. Более поздние версии ядра Linux перешли на схему блокировки с повышенным уровнем детализации, поэтому производительность четырехпроцессорной системы существенно приблизилась к идеальному четырехкратному показателю однопроцессорной системы из-за гораздо меньшего количества конфликтов.

Одна из проблем с детально проработанными схемами блокировки состоит в том, что иногда, чтобы защитить все данные в операции, нужно заблокировать сразу несколько мьютексов. Ранее уже упоминалось, что в некоторых случаях правильное решение заключается в увеличении степени детализации данных, охватываемых мьютексами, так что необходимо заблокировать только один мьютекс. Но такой подход нежелателен, например, когда мьютексы защищают отдельные экземпляры класса. В этом случае блокировка на следующем, более высоком уровне будет означать либо возложение решения о блокировке на пользователя, либо использование единого мьютекса, защищающего все экземпляры этого класса, но ни то ни другое нельзя признать удачным решением.

Если в итоге для той или иной операции придется заблокировать два или более мьютекса, может возникнуть еще одна потенциальная запутанная проблема — *взаимная блокировка*. Она вызвана обстоятельствами, практически противоположными состоянию гонки: потоки не состязаются за первенство, а каждый из них ждет завершения выполнения операции другим, поэтому никакой подвижки не наблюдается ни в одном из потоков.

3.2.4. Взаимная блокировка: проблема и решение

Представьте себе игрушку, к примеру барабан с палочками. Играть с ним можно только при наличии обеих частей, из которых он состоит. А теперь представьте двух малышек, желающих с ним поиграть. Если у одного из них будет и барабан, и палочки, он сможет весело играть, пока не надоеет. Если другому тоже захочется поиграть, ему, как ни досадно, придется подождать. Допустим, барабан и палочки валяются по отдельности в коробке с игрушками, а обоим малышкам вдруг захотелось поиграть на барабане и они стали рыться в ней. Один нашел барабан, а другой — палочки. Возникла тупиковая ситуация: пока кто-нибудь не уступит и не даст поиграть другому, каждый останется при своем, требуя отдать ему недостающее, при этом никто не сможет играть на барабане.

Теперь представьте, что спорят не малышки из-за игрушек, а потоки из-за блокировок мьютексов: чтобы выполнить некую операцию, каждая пара потоков нуждается в блокировке каждой пары мьютексов, у каждого потока имеется один заблокированный мьютекс и он ожидает разблокировки другого мьютекса. Продолжить выполнение не может ни один из потоков, поскольку каждый ждет, когда другой разблокирует свой мьютекс. Такой сценарий называется *взаимной блокировкой*.

и представляет собой самую серьезную проблему при необходимости заблокировать для выполнения одной операции два мьютекса и более.

Общий совет по обходу взаимной блокировки заключается в постоянной блокировке двух мьютексов в одном и том же порядке: если всегда блокировать мьютекс А перед блокировкой мьютекса Б, то взаимной блокировки никогда не произойдет. Иногда это условие выполнить несложно, поскольку мьютексы служат разным целям, но кое-когда все гораздо сложнее, например, когда каждый из мьютексов защищает отдельный экземпляр одного и того же класса. Рассмотрим, к примеру, операцию обмена данными между двумя экземплярами одного и того же класса. Чтобы обеспечить корректный обмен данными и при этом избежать влияния изменений, вносимых в режиме конкурентности, следует заблокировать мьютексы на обоих экземплярах. Но если выбрать определенный порядок, например сначала заблокировать мьютекс для экземпляра, переданного в качестве первого параметра, а затем мьютекс для экземпляра, переданного в качестве второго параметра, то можно получить обратный эффект: стоит всего лишь двум потокам попытаться выполнить обмен данными между теми же двумя экземплярами с переставленными местами параметрами, и вы получите взаимную блокировку!

К счастью, в стандартной библиотеке C++ есть средство от этого в виде `std::lock` — функции, способной одновременно заблокировать два и более мьютекса, не рискуя вызвать взаимную блокировку. Пример из листинга 3.6 показывает, как применить эту функцию для простой операции обмена.

Листинг 3.6. Использование функций `std::lock()` и `std::lock_guard` в операции обмена

```
class some_big_object;
void swap(some_big_object& lhs, some_big_object& rhs);
class X
{
private:
    some_big_object some_detail;
    std::mutex m;
public:
    X(some_big_object const& sd):some_detail(sd){}
    friend void swap(X& lhs, X& rhs)
    {
        if(&lhs==&rhs)
            return;
        std::lock(lhs.m, rhs.m); ← ❶
        std::lock_guard<std::mutex> lock_a(lhs.m, std::adopt_lock); ← ❷
        std::lock_guard<std::mutex> lock_b(rhs.m, std::adopt_lock); ← ❸
        swap(lhs.some_detail, rhs.some_detail);
    }
};
```

Сначала аргументы проверяются на принадлежность к разным экземплярам, поскольку попытки заблокировать уже заблокированный `std::mutex` приводят к неопределенному поведению. (Мьютекс, допускающий несколько блокировок одним потоком, предоставляется в форме `std::recursive_mutex`. Более подробно он рассматривается в подразделе 3.3.3.) Затем вызов `std::lock()` ❶ блокирует два

мьютекса и создаются два экземпляра `std::lock_guard` ❷ и ❸, по одному для каждого мьютекса. В дополнение к мьютексу предоставляется параметр `std::adopt_lock`, чтобы показать объектам `std::lock_guard`, что мьютексы уже заблокированы. Объекты должны овладеть существующей блокировкой мьютекса, а не пытаться заблокировать его в конструкторе.

Тем самым обеспечивается корректная разблокировка мьютекса при выходе из функции в общем случае, когда защищенная операция может выдать исключение, а также разрешается простое возвращение управления. Следует также отметить, что блокировка либо `lhs.m`, либо `rhs.m` внутри вызова `std::lock` может привести к выдаче исключения, в таком случае исключение распространяется из `std::lock`. Если функцией `std::lock` успешно заблокирован один мьютекс, а исключение выдано при попытке заблокировать другой, первый мьютекс разблокируется автоматически: в отношении блокировки предоставленных мьютексов функция `std::lock` обеспечивает семантику «все или ничего».

В C++17 предоставляется дополнительная поддержка этого сценария в виде нового RAII-шаблона `std::scoped_lock<>`. Он практически эквивалентен `std::lock_guard<>`, за исключением того, что является *вариационным шаблоном*, принимающим в качестве параметров шаблона список типов мьютексов, а в качестве аргументов конструктора — список мьютексов. Предоставленные конструктору мьютексы блокируются с использованием такого же алгоритма, как и в `std::lock`, и, когда конструктор завершает работу, они оказываются заблокированными, а затем разблокируются в деструкторе. Операцию `swap()` из листинга 3.6 можно переписать следующим образом:

```
void swap(X& lhs, X& rhs)
{
    if(&lhs==&rhs)
        return;
    std::scoped_lock guard(lhs.m,rhs.m); ←❶
    swap(lhs.some_detail,rhs.some_detail);
}
```

В этом примере используется еще одна возможность, добавленная в C++17, — автоматическое выведение параметров шаблона класса. При наличии компилятора C++17, который, скорее всего, будет применяться при использовании `std::scoped_lock`, механизм подразумеваемого выведения параметра шаблона класса, имеющийся в C++17, выберет подходящие типы мьютексов из типов объектов, переданных конструктору в объекте ❶. Эта строка кода эквивалентна полномасштабной версии:

```
std::scoped_lock<std::mutex,std::mutex> guard(lhs.m,rhs.m);
```

Наличие шаблона `std::scoped_lock` означает, что в большинстве случаев там, где до выхода C++17 приходилось использовать `std::lock`, теперь можно будет задействовать `std::scoped_lock`, сократив поле возможностей допущения ошибок, что можно только приветствовать!

Применение `std::lock` и `std::scoped_lock<>` позволяет избавиться от взаимных блокировок, когда нужно завладеть сразу двумя и более блокировками, однако оно

не поможет, если блокировки захватываются разобщенно. В таком случае, чтобы гарантировать обход взаимных блокировок, разработчикам приходится полагаться на самодисциплину. А это не так-то просто: взаимоблокировки относятся к одной из самых неприятных проблем, с которой приходится сталкиваться в многопоточном коде, их возникновение зачастую невозможно предсказать, поскольку в большинстве случаев все работает нормально. И тем не менее существует ряд относительно простых правил, помогающих создавать код, не подверженный взаимным блокировкам.

3.2.5. Дополнительные рекомендации по обходу взаимных блокировок

Чаще всего причинами взаимных блокировок становятся именно блокировки, но это не единственное условие их возникновения. Они могут проявиться при конкурентном выполнении двух потоков и отсутствии блокировок, когда каждый из потоков вызывает функцию `join()` в отношении объекта `std::thread` другого потока. В таком случае дальнейшее выполнение кода невозможно ни в одном из потоков, поскольку каждый из них ожидает завершения работы функции в другом потоке, уподобляясь детям, отнимающим друг у друга игрушки. Простой цикл, когда поток будет ожидать выполнения какого-либо действия другим потоком, а тот одновременно ожидать завершения действия первым потоком, может возникнуть где угодно. И дело может не ограничиться двумя потоками: взаимную блокировку способен вызвать и цикл из трех и более потоков. Все рекомендации по обходу взаимных блокировок сводятся к одному: не ждать завершения операции другим потоком, если есть вероятность, что он также ждет завершения операции текущим потоком. Способы определения и устранения вероятности ожидания другим потоком завершения операции текущим потоком существуют в виде отдельных рекомендаций.

Избегайте вложенных блокировок

И первой рекомендацией будет самая простая: не устанавливайте блокировку, если уже есть какая-либо блокировка. Если придерживаться данной рекомендации, то получить взаимную блокировку по причине использования одних лишь блокировок станет невозможно, поскольку каждый поток будет удерживать только одну блокировку. При этом сохранится вероятность взаимной блокировки по другим причинам, например из-за взаимного ожидания потоками завершения операции. Но, вероятно, самыми распространенными причинами взаимных блокировок будут все же блокировки мьютексов. Если нужно получить несколько блокировок, то с помощью `std::lock` их следует заключить в рамки единого действия, чтобы избежать взаимных блокировок.

При удержании блокировки вызова избегайте кода, предоставленного пользователем

Это простое следствие из предыдущей рекомендации. Поскольку код предоставляется пользователем, неизвестно, что он делает: он может заниматься чем угодно, в том числе устанавливать блокировку. Если при удержании блокировки вызвать

пользовательский код, устанавливающий блокировку, окажется нарушена рекомендация, предписывающая избегать вложенных блокировок, и может возникнуть взаимная блокировка. Но иногда подобное неизбежно: если вы создаете общий код, например стек из подраздела 3.2.3, то код, предоставляемый пользователем, используется при каждой операции с типом или типами параметров. В таком случае нужна новая рекомендация.

Устанавливайте блокировки в фиксированном порядке

Если есть настоятельная необходимость установить две и более блокировки, но в рамках единой операции с помощью `std::lock` это невозможно, лучшее, что можно сделать, — установить их в каждом потоке в одном и том же порядке. В подразделе 3.2.4 этот прием уже рассматривается как один из способов, позволяющих избежать взаимной блокировки при блокировке двух мьютексов, — главное, определить единый порядок, распространяющийся на все потоки. В некоторых случаях сделать это довольно легко. Посмотрите, к примеру, на стек из подраздела 3.2.3 — мьютекс является внутренним для каждого экземпляра стека, а вот операции над записями данных, сохраненными в стеке, требуют вызова кода, предоставленного пользователем. Можно, конечно, добавить ограничение, согласно которому ни одна из операций над элементами данных, хранящимися в стеке, не должна выполнять никаких операций над самим стеком. Это становится дополнительной обузой для пользователя стека, но обстоятельства, при которых данные, хранящиеся в контейнере, получают доступ к этому самому контейнеру, возникают крайне редко, и это довольно заметное событие, так что пользователь способен справиться с ним вполне успешно.

Но бывают случаи, когда не все так просто, как при исследовании операции обмена из подраздела 3.2.4. По крайней мере в этом случае мьютексы можно заблокировать одновременно, но такая возможность предоставляется не всегда. Если вернуться к связанному списку из раздела 3.1, можно увидеть, что одним из способов защиты списка является установка мьютекса на каждом узле. Затем, чтобы получить доступ к списку, потоки должны устанавливать блокировку на каждый интересующий их узел. Чтобы поток удалил запись, он должен установить блокировку на три узла: удаляемый и узлы с обеих сторон от него, поскольку все они так или иначе изменяются. Также, проходя по списку, поток должен удерживать блокировку на текущем узле, устанавливая ее на следующем по порядку узле, чтобы гарантировать, что в это же время не будет изменен следующий указатель. После установки блокировки на следующем узле с первого узла ее можно снять, поскольку надобность в ней уже отпала.

Такой эстафетный стиль блокировки позволяет нескольким потокам обращаться к списку при условии, что все они обращаются к разным узлам. Но во избежание взаимной блокировки узлы неизменно должны блокироваться в одном и том же порядке: если два узла пытаются пройти по списку в противоположных направлениях, используя эстафетные блокировки, то в середине списка они могут заблокировать друг друга. Если узлы А и Б — соседи в списке, то поток, проходящий в одном направлении, будет пытаться удерживать блокировку на узле А, а также установить блокировку на узле Б. Поток, проходящий в другом направлении, станет удерживать

блокировку узла Б и пытаться установить блокировку на узле А. Возникнет классический сценарий взаимной блокировки (рис. 3.2).

Поток 1	Поток 2
Блокировка мьютекса главной записи	
Чтение указателя головного узла	
Блокировка мьютекса головного узла	
Разблокировка мьютекса главной записи	
	Блокировка мьютекса главной записи
Чтение указателя от головного к следующему узлу	Блокировка мьютекса хвостового узла
Блокировка мьютекса следующего узла	Чтение указателя от хвостового к предыдущему узлу
Чтение указателя от следующего к очередному узлу	Разблокировка мьютекса хвостового узла
...	...
Блокировка мьютекса узла А	Блокировка мьютекса узла В
Чтение указателя от А к следующему узлу (которым является узел Б)	Чтение указателя от узла В к следующему узлу (которым является узел Б)
	Блокировка мьютекса узла Б
Попытка блокировки мьютекса узла Б	Разблокировка мьютекса узла В
	Чтение указателя от узла Б к следующему узлу (которым является узел А)
	Попытка блокировки мьютекса узла А
Взаимная блокировка!	

Рис. 3.2. Взаимная блокировка потоков, проходящих по списку в противоположных направлениях

Если при удалении узла Б, расположенного между узлами А и В, поток устанавливает блокировку на узле Б до установки блокировки на узлах А и В, то возникает вероятность взаимной блокировки с потоком, проходящим по списку. Такой поток сначала попытается заблокировать либо узел А, либо узел В в зависимости от направления прохода, но потом обнаружит, что не может установить блокировку на узле Б, поскольку она удерживается потоком, выполняющим удаление и пытающимся установить блокировки на узлах А и В.

Один из способов обхода такой взаимной блокировки заключается в определении порядка прохода по списку, чтобы поток всегда был вынужден блокировать А до бло-

кировки Б и Б — до блокировки В. Это устранил вероятность взаимной блокировки цепой запрещения прохода списка в обратном направлении. Такое же соглашение должно зачастую устанавливаться и для других структур данных.

Используйте иерархию блокировок

Являясь частным случаем определения порядка блокировок, иерархия блокировок позволяет обеспечить средство проверки соблюдения соглашения в ходе выполнения программы. Замысел заключается в том, чтобы разбить приложение на уровни и идентифицировать все мьютексы, которые могут быть заблокированы на любом заданном уровне. Когда код пытается заблокировать мьютекс, он не может этого сделать, если в нем уже содержится блокировка из нижнего уровня. Это можно проверить в ходе выполнения программы, назначив номера уровней каждому мьютексу и сохранив записи о том, какие мьютексы заблокированы каждым потоком. Этот шаблон получил очень широкое распространение, но его прямая поддержка в стандартной библиотеке C++ не обеспечивается, поэтому нужно создать собственный тип мьютекса `hierarchical_mutex`, код которого приведен в листинге 3.8.

В листинге 3.7 показаны два потока, задействующих иерархический мьютекс.

Листинг 3.7. Использование иерархии блокировок во избежание взаимной блокировки

```
hierarchical_mutex high_level_mutex(10000); ← ❶
hierarchical_mutex low_level_mutex(5000); ← ❷
hierarchical_mutex other_mutex(6000); ← ❸ ❹
int do_low_level_stuff();
int low_level_func()
{
    std::lock_guard<hierarchical_mutex> lk(low_level_mutex); ← ❺
    return do_low_level_stuff();
}
void high_level_stuff(int some_param);
void high_level_func()
{
    std::lock_guard<hierarchical_mutex> lk(high_level_mutex); ← ❻
    high_level_stuff(low_level_func()); ← ❽
}
void thread_a() ← ❼
{
    high_level_func();
}

void do_other_stuff();
void other_stuff()
{
    high_level_func(); ← ❿
    do_other_stuff();
}
void thread_b() ← Ⓢ
{
    std::lock_guard<hierarchical_mutex> lk(other_mutex); ← Ⓣ
    other_stuff();
}
```

В этом коде имеется три экземпляра `hierarchical_mutex`, ❶, ❷ и ❸, созданные с убывающими номерами иерархии уровней. Поскольку механизм определен таким образом, что при удержании блокировки `hierarchical_mutex` можно установить блокировку только `hierarchical_mutex` с меньшим номером уровня иерархии, это ограничивает возможности кода.

Если предположить, что `do_low_level_stuff` не блокирует никакие мьютексы, то `low_level_func` является дном вашей иерархии и блокирует `low_level_mutex` ❹. Функция `high_level_func` вызывает `low_level_func` ❸, удерживая блокировку `high_level_mutex` ❺, но это вполне допустимо, учитывая, что иерархический уровень мьютекса `high_level_mutex` (❶: 10000) выше, чем мьютекса `low_level_mutex` (❷: 5000).

Поток `thread_a()` ❷ соблюдает правила, поэтому работает нормально.

А вот поток `thread_b()` ❸ нарушает правила, что дает сбой в ходе выполнения. Прежде всего он блокирует мьютекс `other_mutex` ❾, значение иерархического уровня которого всего 6000 ❸. Это означает, что он должен располагаться где-то в середине иерархии. Когда `other_stuff()` вызывает `high_level_func()` ❿, это нарушает иерархию: `high_level_func()` пытается установить блокировку на мьютексе `high_level_mutex` со значением 10000, значительно большим, чем текущее значение уровня иерархии 6000. Поэтому экземпляр класса `hierarchical_mutex` выдаст отчет об ошибке, возможно, путем выдачи исключения или прерывания выполнения программы. Взаимные блокировки между иерархическими мьютексами невозможны, поскольку порядок блокировки обеспечивается самими мьютексами. Это означает невозможность одновременного удержания двух блокировок, если они относятся к одному и тому же иерархическому уровню, и эстафетная схема блокировки требует, чтобы каждый мьютекс в цепочке имел меньшее значение иерархического уровня, чем предыдущий. Однако в некоторых случаях это условие может оказаться неподходящим.

Этот пример показывает и еще одну возможность — применение шаблона `std::lock_guard<>` с типом мьютекса, определенным пользователем. Класс `hierarchical_mutex` не является частью стандарта, но его создание не представляет особой трудности, простая реализация показана в листинге 3.8. Хотя это тип, определенный пользователем, он может использоваться с `std::lock_guard<>`, поскольку в нем реализованы три компонентные функции, необходимые для соответствия концепции мьютекса: `lock()`, `unlock()` и `try_lock()`. Непосредственное применение `try_lock()` еще не было показано, но в нем нет ничего сложного: если блокировка мьютекса удерживается другим потоком, эта функция возвращает `false`, а не ждет, пока вызвавший ее поток сможет установить блокировку на мьютексе. Она также может использоваться внутри `std::lock()` в качестве части алгоритма предотвращения взаимных блокировок.

В реализации `hierarchical_mutex` задействуется локальная по отношению к потоку переменная, предназначенная для хранения текущего значения уровня иерархии. Данное значение доступно всем экземплярам мьютексов, но оно разное для каждого из потоков. Это позволяет коду проверять поведение каждого потока по отдельности, и код для каждого мьютекса может проверять, разрешено ли текущему потоку заблокировать этот мьютекс или нет.

Листинг 3.8. Простой иерархический мьютекс

```

class hierarchical_mutex
{
    std::mutex internal_mutex;
    unsigned long const hierarchy_value;
    unsigned long previous_hierarchy_value;
    static thread_local unsigned long this_thread_hierarchy_value; ← ❶
    void check_for_hierarchy_violation()
    {
        if(this_thread_hierarchy_value <= hierarchy_value) ← ❷
        {
            throw std::logic_error("mutex hierarchy violated");
        }
    }
    void update_hierarchy_value()
    {
        previous_hierarchy_value=this_thread_hierarchy_value; ← ❸
        this_thread_hierarchy_value=hierarchy_value;
    }
public:
    explicit hierarchical_mutex(unsigned long value):
        hierarchy_value(value),
        previous_hierarchy_value(0)
    {}
    void lock()
    {
        check_for_hierarchy_violation(); ← ❹
        internal_mutex.lock(); ← ❺
        update_hierarchy_value(); ← ❻
    }
    void unlock()
    {
        if(this_thread_hierarchy_value!=hierarchy_value) ← ❼
            throw std::logic_error("mutex hierarchy violated"); ← ❽
        this_thread_hierarchy_value=previous_hierarchy_value; ← ❾
        internal_mutex.unlock();
    }
    bool try_lock()
    {
        check_for_hierarchy_violation();
        if(!internal_mutex.try_lock()) ← ❼
            return false;
        update_hierarchy_value();
        return true;
    }
};
thread_local unsigned long
    hierarchical_mutex::this_thread_hierarchy_value(ULONG_MAX); ← ❻

```

Главное здесь заключается в использовании значения `thread_local`, представляющего собой значение уровня иерархии текущего потока `this_thread_hierarchy_value` ❶. Оно инициализируется максимальным значением ❷, поэтому изначально

заблокирован может быть любой мьютекс. Поскольку это значение объявлено как `thread_local`, собственная копия может быть у каждого потока, поэтому состояние переменной в одном потоке совершенно не зависит от состояния переменной, когда чтение происходит из другого потока. Дополнительные сведения о `thread_local` приводятся в разделе A.8.

Итак, при первой блокировке экземпляра `hierarchical_mutex` значением `this_thread_hierarchy_value` является `ULONG_MAX`. По своей природе это значение больше любого другого, поэтому проверка в `check_for_hierarchy_violation()` ❷ проходит успешно. Раз это происходит, `lock()` делегируется внутреннему мьютексу для блокировки ❹. После успешной установки этой блокировки перархическое значение можно обновить ❸.

Если теперь заблокировать *другой* экземпляр `hierarchical_mutex`, удерживая блокировку на первом экземпляре, в значении `this_thread_hierarchy_value` отразится перархическое значение первого мьютекса. Чтобы пройти проверку ❷, значение уровня перархии этого второго мьютекса теперь должно быть меньше значения уровня перархии уже удерживаемого мьютекса.

Теперь важно сохранить предыдущее значение уровня перархии для текущего потока, чтобы можно было восстановить его в `unlock()` ❻, в противном случае не получится снова заблокировать мьютекс с более высоким значением уровня перархии, даже если поток не удерживает никаких блокировок. Поскольку предыдущее значение уровня перархии сохраняется, только когда удерживается внутренний мьютекс `internal_mutex` ❺, и его восстановление выполняется до разблокировки внутреннего мьютекса ❻, появляется возможность безопасно хранить его в самом объекте `hierarchical_mutex`, так как он надежно защищен блокировкой внутреннего мьютекса. Во избежание перархической путаницы из-за разблокировки в неподходящем порядке в строке ❾ выдается ошибка, если разблокированный мьютекс не является самым последним заблокированным мьютексом. Возможно применение и других механизмов, но этот самый простой. Функция `try_lock()` работает так же, как и функция `lock()`, за исключением того, что если вызов `try_lock()` в отношении внутреннего мьютекса `internal_mutex` дает сбой ❷, значит, блокировкой владеет другой поток, поэтому обновления значения уровня перархии не происходит и вместо `true` возвращается `false`.

Поскольку обнаружение относится к проверкам времени выполнения, оно по крайней мере не зависит от времени — не приходится ждать неких редко возникающих условий, вызывающих взаимную блокировку. К тому же процесс разработки, требующий такого разделения приложения и мьютексов, помогает избавиться от многочисленных вероятностей взаимных блокировок еще до того, как будет создан соответствующий код. Возможно, стоит проработать проект в этом направлении, даже если дело еще не дошло для написания проверок времени выполнения.

Выход рекомендаций за пределы блокировок

В начале данного раздела уже упоминалось, что взаимные блокировки возникают не только при использовании блокировок — они возможны при любой конструкции синхронизации, способной привести к циклу ожидания. Следовательно, рекоменда-

ции нужно распространить и на эти случаи. Например, точно так же, как по возможности стоит избегать вложенных блокировок, не следует дожидаться, когда поток станет выполнять операцию, удерживая при этом блокировку, так как этому потоку, чтобы продолжать выполняться, может потребоваться эту блокировку установить. Аналогично этому, намереваясь дожидаться завершения потока, возможно, стоит определить иерархию потоков, чтобы поток ожидал завершения только тех потоков, которые находятся ниже его по иерархии. Для этого есть очень простой способ — в соответствии с описаниями, приведенными в подразделе 3.1.2 и разделе 3.3, обеспечить объединение потоков в той же функции, которая их запустила.

При разработке кода с прицелом на избавление от взаимных блокировок можно рассчитывать на то, что с помощью `std::lock()` и `std::lock_guard` удастся исключить большинство случаев их возникновения при простых блокировках. Но иногда требуется более гибкий подход. Для таких случаев стандартной библиотекой предоставляется шаблон `std::unique_lock`. Подобно `std::lock_guard`, он представляет собой шаблон класса, параметризованный типом мьютекса и обеспечивающий управление блокировками в том же RAII-стиле, что и `std::lock_guard`, но с немного более гибким подходом.

3.2.6. Гибкая блокировка с применением `std::unique_lock`

Шаблон `std::unique_lock` обеспечивает немного более гибкий подход, по сравнению с `std::lock_guard`, за счет смягчения инвариантов: экземпляр `std::unique_lock` не всегда владеет связанным с ним мьютексом. Начнем с того, что конструктору в качестве второго аргумента можно передавать не только объект `std::adopt_lock`, заставляющий объект блокировки управлять блокировкой мьютекса, но и объект отсрочки блокировки `std::defer_lock`, показывающий, что мьютекс при конструировании должен оставаться разблокированным. Блокировку можно установить позже, вызвав функцию `lock()` для объекта `std::unique_lock` (но *не* мьютекса) или же передав объект `std::unique_lock` функции `std::lock()`. Код листинга 3.6 можно легко переписать так, как показано в листинге 3.9, воспользовавшись `std::unique_lock` и `std::defer_lock` ^❶ вместо `std::lock_guard` и `std::adopt_lock`. Код имеет то же количество строк и является эквивалентным, за исключением небольшой особенности: `std::unique_lock` занимает немного большее пространство и действует несколько медленнее по сравнению с `std::lock_guard`. За гибкость, заключающуюся в разрешении экземпляру `std::unique_lock` не владеть мьютексом, приходится расплачиваться тем, что эта информация должна быть сохранена и обновлена.

Листинг 3.9. Использование `std::lock()` и `std::unique_lock` в операции обмена

```
class some_big_object;
void swap(some_big_object& lhs, some_big_object& rhs);
class X
{
private:
    some_big_object some_detail;
```

```

    std::mutex m;
public:
    X(some_big_object const& sd):some_detail(sd){}
    friend void swap(X& lhs, X& rhs)
    {
        if(&lhs==&rhs)
            return;
        std::unique_lock<std::mutex> lock_a(lhs.m,std::defer_lock);
        std::unique_lock<std::mutex> lock_b(rhs.m,std::defer_lock);
        std::lock(lock_a,lock_b);
        swap(lhs.some_detail,rhs.some_detail);
    }
};

```

Поскольку экземпляр `std::unique_lock` в листинге 3.9 предоставляет компонентные функции `lock()`, `try_lock()` и `unlock()`, объекты `std::unique_lock` можно передать `std::lock()` ②. В них происходит перенаправление к компонентным функциям с таким же именем в основном мьютексе, где выполняется работа и обновляется флаг внутри экземпляра `std::unique_lock`, показывающий, принадлежит ли мьютекс в данный момент этому экземпляру. Этот флаг необходим, чтобы гарантировать правильный вызов функции `unlock()` в деструкторе. Если экземпляр *действительно* владеет мьютексом, деструктор *должен* вызвать функцию `unlock()`, в ином случае — *не должен*. Этот флаг можно запросить, вызвав компонентную функцию `owns_lock()`. Если передача владения блокировкой или какие-то другие действия, требующие `std::unique_lock`, не предусматриваются, лучше все же по возможности воспользоваться вариационным шаблоном `std::scoped_lock` из C++17 (см. подраздел 3.2.4).

Как и следует ожидать, этот флаг должен где-нибудь храниться. Поэтому размер объекта `std::unique_lock` обычно превышает размер объекта `std::lock_guard` и, кроме того, использование `std::unique_lock` вместо `std::lock_guard` немного ухудшает производительность, поскольку флаг нуждается в обновлении или проверке. Если `std::lock_guard` вполне вас устраивает, рекомендуется отдавать предпочтение именно этому объекту. И тем не менее бывают случаи, когда `std::unique_lock` лучше вписывается в решение текущей задачи в силу необходимости дополнительной гибкости. В качестве одного примера можно привести уже рассмотренную отложенную блокировку, а в качестве другого — необходимость передачи владения блокировкой из одной области видимости в другую.

3.2.7. Передача владения мьютексом между областями видимости

Поскольку экземпляры `std::unique_lock` не обязаны владеть связанными с ними мьютексами, владение мьютексом может передаваться между экземплярами путем *перемещения*. В некоторых случаях, например при возвращении экземпляра из функции, оно происходит автоматически, а в других случаях его необходимо выполнять явным образом вызовом функции `std::move()`. По сути, все зависит от того, является ли источник *l-значением* — реальной переменной или ссылкой на

такую — или *r-значением* — неким временным объектом. Владение передается автоматически, если источник является *r-значением*, или же должно передаваться явным образом, если он является *l-значением*, во избежание случайной передачи владения за пределы переменной. Класс `std::unique_lock` — это пример *перемещаемого*, но не *копируемого* типа. Дополнительные сведения о семантике приводятся в подразделе A.1.1.

Один из вариантов возможного использования заключается в разрешении функции заблокировать мьютекс, а затем передать владение этой блокировкой вызывающему коду, который впоследствии сможет выполнить дополнительные действия под защитой этой же самой блокировки. Соответствующий пример показан в следующем фрагменте кода, где функция `get_lock()` блокирует мьютекс, а затем подготавливает данные перед тем, как вернуть блокировку вызывающему коду:

```
std::unique_lock<std::mutex> get_lock()
{
    extern std::mutex some_mutex;
    std::unique_lock<std::mutex> lk(some_mutex);
    prepare_data();
    return lk;      ← ❶
}
void process_data()
{
    std::unique_lock<std::mutex> lk(get_lock()); ← ❷
    do_something();
}
```

Поскольку `lk` — автоматическая переменная, объявленная внутри функции, ее можно возвратить напрямую ❶, без вызова функции `std::move()`. О вызове конструктора перемещения позаботится компилятор. Затем функция `process_data()` сможет передать владение непосредственно в собственный экземпляр `std::unique_lock` ❷, а вызов функции `do_something()` может полагаться на правильную подготовку данных без их изменения в промежутке времени со стороны другого потока.

Обычно такой шаблон следует применять, когда блокируемый мьютекс зависит от текущего состояния программы или от аргумента, переданного в функцию, возвращающую объект `std::unique_lock`. Например, так делается, когда блокировка не возвращается напрямую, а является компонентными данными шлюзового класса, используемого для обеспечения корректного доступа к неким заблокированным защищенным данным. В таком случае доступ к данным осуществляется через этот шлюзовый класс: когда нужно получить доступ к данным, приобретается экземпляр шлюзового класса (вызовом функции наподобие `get_lock()`, как в предыдущем примере), который получает блокировку. Затем к данным можно будет обращаться через компонентные функции шлюзового объекта. Когда необходимость в шлюзовом объекте отпадает, его уничтожают, в результате чего снимается блокировка и доступ к защищенным данным разрешается *другим* потокам. Такой шлюзовый объект вполне можно сделать перемещаемым, для того чтобы его можно было возвращать из функции, и в таком случае компонентные данные объекта блокировки также должны быть перемещаемыми.

Гибкость, присущая классу `std::unique_lock`, позволяет экземплярам снимать их блокировки до того, как они будут удалены. Сделать это можно с помощью компонентной функции `unlock()` точно так же, как это происходит с мьютексом. Класс `std::unique_lock` поддерживает точно такой же основной набор компонентных функций для блокировки и разблокировки, как и для мьютекса, поэтому его можно использовать с такими обобщенными функциями, как `std::lock`. Возможность снятия блокировки до удаления экземпляра `std::unique_lock` означает допустимость ее выборочного снятия в конкретной ветви кода, если окажется, что она больше не нужна. Это может положительно повлиять на производительность приложения, поскольку удержание блокировки сверх требуемого времени скорее ухудшит производительность из-за того, что снятия блокировки могут ожидать другие потоки и их работа продлится дольше, чем нужно на самом деле.

3.2.8. Блокировка с соответствующей степенью детализации

О степени детализации блокировки уже говорилось в подразделе 3.2.3: это надуманное понятие, касающееся объема данных, защищенного отдельно взятой блокировкой. Блокировка с более подробной детализацией защищает незначительный объем данных, а с менее подробной — большой. Важно не только выбрать детализацию блокировки, которой окажется достаточно для защиты нужных данных, но и обеспечить, чтобы блокировка удерживалась только для тех операций, которым она требуется. Каждому знакомо чувство раздражения, возникающее, когда ты стоишь с тележкой, полной продуктов, в очереди к кассе, а тот, кто рассчитывается, вдруг вспоминает, что забыл взять клюквенный соус, и заставляет всех ждать, пока он за ним сходит. И мы можем предположить, что чувствует кассир, когда он уже готов принять оплату, а клиент только в этот момент начинает искать в сумке конфект. Ситуация становится куда менее напряженной, если каждый подвозит к кассе все, что хотел, и готов нужным способом оплатить товар.

То же самое применимо и к потокам: если в ожидании одного и того же ресурса находятся сразу несколько потоков (кассиров на рабочем месте) и любой из потоков удерживает блокировку дольше необходимого, возрастает общее время ожидания (не стоит дожидаться подхода своей очереди к кассе, чтобы приступить к покупке клюквенного соуса). По возможности следует блокировать мьютекс только при получении доступа к совместно используемым данным и постараться выполнять любую другую обработку данных вне блокировки. В частности, удерживая блокировку, не следует осуществлять требующие много времени действия наподобие файлового ввода-вывода. Обычно подобный ввод-вывод в сотни, если не в тысячи раз медленнее чтения или записи сопоставимого объема данных из памяти. Если блокировка не предназначена для защиты доступа к файлу, ввод-вывод при удерживаемой блокировке приведет к неоправданной задержке работы других потоков, так

как они будут заблокированы в ожидании получения блокировки, что, возможно, сведет на нет весь выигрыш в производительности, полученный в результате использования нескольких потоков.

В подобной ситуации выгодно задействовать класс `std::unique_lock`, поскольку он позволяет вызвать функцию `unlock()`, когда коду больше не нужно обращаться к совместно используемым данным, а затем опять вызвать функцию `lock()`, если позже в коде это потребуется:

```
void get_and_process_data()
{
    std::unique_lock<std::mutex> my_lock(the_mutex);
    some_class data_to_process=get_next_data_chunk();
    my_lock.unlock();
    result_type result=process(data_to_process);
    my_lock.lock();
    write_result(data_to_process,result);
}
```

1 Заблокированный мьютекс при вызове `process()` не нужен

2 Повторная блокировка мьютекса для записи результата

При вызове функции `process()` заблокированный мьютекс не нужен, поэтому перед вызовом 1 он преднамеренно разблокируется, а потом снова блокируется 2.

Надеюсь, вполне очевидно, что при наличии одного мьютекса защита всей структуры данных не только повышает вероятность блокировки, но и снижает вероятность сокращения времени ее удержания. Последующие этапы операции потребуют блокировки того же мьютекса, поэтому она должна удерживаться дольше. Такое двойное удорожание блокировки является также двойным стимулом для перехода там, где это возможно, к более детализированной блокировке.

Как показывает данный пример, блокировка с соответствующей детализацией касается не только объема блокируемых данных, но и продолжительности удержания блокировки и того, какие операции выполняются в ходе нее. *В основном блокировка должна удерживаться в течение минимально возможного времени, необходимого для выполнения требуемых операций.* Это также означает, что затраты по времени операции, например получение еще одной блокировки (даже если известно, что это не приведет к взаимной блокировке) или ожидание завершения ввода-вывода, не должны выполняться при удерживании блокировки, если это не вызвано крайней необходимостью.

В листингах 3.6 и 3.9 блокировка двух мьютексов требовала операции обмена данными, которой, что вполне очевидно, нужен был конкурентный доступ к обоим объектам. Предположим, что вместо этого понадобилось провести операцию сравнения с участием простого компонентного элемента данных, представляющего собой обычный целочисленный тип `int`. Изменит ли это сложившуюся ситуацию? Значения типа `int` не создают существенных издержек при копировании, поэтому можно легко скопировать данные каждого сравниваемого объекта, удерживая его блокировку, а затем сравнить скопированные значения. Это означало бы, что по времени блокировка удерживается на каждом мьютексе минимально, а также что одна блокировка не удерживается при удержании другой. В листинге 3.10

показаны класс `Y`, в котором все именно так и происходит, и примерная реализация оператора сравнения.

Листинг 3.10. Блокировка только одного из двух мьютексов с разном их блокировки по времени в операторе сравнения

```
class Y
{
private:
    int some_detail;
    mutable std::mutex m;
    int get_detail() const
    {
        std::lock_guard<std::mutex> lock_a(m); ←❶
        return some_detail;
    }
public:
    Y(int sd):some_detail(sd){}
    friend bool operator==(Y const& lhs, Y const& rhs)
    {
        if(&lhs==&rhs)
            return true;
        int const lhs_value=lhs.get_detail(); ←❷
        int const rhs_value=rhs.get_detail(); ←❸
        return lhs_value==rhs_value; ←❹
    }
};
```

В данном случае оператор сравнения сначала извлекает сравниваемые значения путем вызова компонентной функции `get_detail()` ❷ и ❸. Эта функция извлекает значение, защищая его блокировкой ❶. Затем оператор сравнения выполняет свою задачу с извлеченными значениями ❹. Но следует обратить внимание на то, что наряду с сокращением времени удержания блокировки за счет удержания в любой момент времени только одной блокировки и исключения возможности взаимной блокировки была немного изменена семантика, в отличие от операции сравнения, где обе блокировки удерживались одновременно. В листинге 3.10 показано, что, если оператор возвращает значение `true`, это означает, что значение `lhs.some_detail` в какой-то момент времени равно значению `rhs.some_detail` в другой момент времени. Между двумя операциями считывания эти два значения можно как угодно изменить, например, между выполнением кода в строках ❷ и ❸ программа вообще могла бы поменять их местами, что сделало бы сравнение бесполезным. Сравнение на равенство может вернуть значение `true`, чтобы показать, что значения были равны, даже если ни в один из моментов они не были равны. Поэтому, внося подобные изменения, важно действовать осмотрительно, чтобы семантика операции не менялась, вызывая тем самым проблемы: *если не удерживать требуемые блокировки на весь период проведения операции, возникает вероятность гонки*.

Порой подходящей степени детализации просто не существует, поскольку не все обращения к структуре данных требуют одинакового уровня защиты. В таком случае вместо обычного класса `std::mutex` стоит воспользоваться каким-нибудь альтернативным механизмом.

3.3. Альтернативные средства защиты совместно используемых данных

Хотя мьютексы и представляют собой наиболее универсальный механизм, они не единственные игроки на поле защиты совместно используемых данных, существуют и альтернативные варианты, более подходящие в конкретных обстоятельствах.

Одним из крайних, но весьма распространенных случаев является ситуация, при которой совместно используемые данные нуждаются в защите от конкурентного доступа только во время их инициализации и после этого уже не требуют никакой явной синхронизации. Это может быть связано с тем, что данные создаются только для чтения, и поэтому отсутствуют всяческие проблемы синхронизации, или же с тем, что необходимая защита реализуется по умолчанию как часть операций с данными. В любом случае блокировка мьютекса, выполняемая исключительно для защиты инициализации, становится ненужной после того, как данные инициализированы, и будет неоправданно ухудшать производительность. Именно поэтому в стандарте C++ имеется механизм, предназначенный исключительно для защиты совместно используемых данных во время инициализации.

3.3.1. Защита совместно используемых данных во время инициализации

Предположим, что есть совместно используемый ресурс, создание которого настолько затратно, что заниматься этим хочется лишь в крайней необходимости: возможно, он открывает подключение к базе данных или распределяет слишком большой объем памяти. Подобная *отложенная* (или *ленивая*) *инициализация* (lazy initialization) довольно часто встречается в однопоточном коде — каждая операция, требующая ресурса, сначала проверяет, не был ли он инициализирован, и, если не был, прежде чем воспользоваться этим ресурсом, инициализирует его:

```
std::shared_ptr<some_resource> resource_ptr;
void foo()
{
    if(!resource_ptr)
    {
        resource_ptr.reset(new some_resource); ←❶
    }
    resource_ptr->do_something();
}
```

Если совместно используемый ресурс безопасен при получении к нему конкурентного доступа, единственной частью, нуждающейся в защите при преобразовании кода в многопоточный, является инициализация ❶. По слишком простое преобразование, подобное показанному в листинге 3.11, может привести к ненужному последовательному выстраиванию потоков, использующих ресурс. Причина в том, что каждый поток будет вынужден ожидать разблокировки мьютекса, чтобы проверить, не был ли ресурс уже инициализирован.

Листинг 3.11. Безопасная для потоков отложенная инициализация с использованием мьютекса

```

std::shared_ptr<some_resource> resource_ptr;
std::mutex resource_mutex;
void foo()
{
    std::unique_lock<std::mutex> lk(resource_mutex);
    if(!resource_ptr)
    {
        resource_ptr.reset(new some_resource);
    }
    lk.unlock();
    resource_ptr->do_something();
}

```

Здесь все потоки
выстраиваются
в последовательность

В защите нуждается
только инициализация

Этот код настолько распространен, а ненужное выстраивание в последовательность вызывает столько проблем, что многие пытались придумать более подходящий способ решения данной задачи, включая небезызвестный *шаблон блокировки с двойной проверкой*: сначала указатель считывается без получения блокировки (в строке ❶ следующего кода), которая устанавливается, только если он имеет значение NULL. После получения блокировки указатель проверяется *еще раз* (в строке ❷, следовательно, в этой части получается *двойная проверка*) на тот случай, если между первой проверкой и получением блокировки данным потоком инициализация была выполнена каким-нибудь другим потоком:

```

void undefined_behaviour_with_double_checked_locking()
{
    if(!resource_ptr) ←❶
    {
        std::lock_guard<std::mutex> lk(resource_mutex);
        if(!resource_ptr) ←❷
        {
            resource_ptr.reset(new some_resource); ←❸
        }
    }
    resource_ptr->do_something(); ←❹
}

```

К сожалению, эта схема снискала себе дурную славу, так как в ней имеется потенциальная вероятность вложенного состояния гонки, поскольку чтение за пределами блокировки ❶ не синхронизировано с записью, выполняемой другим потоком под блокировкой ❸. Тем самым создается состояние гонки, распространяющееся не только на сам указатель, но и на указываемый объект: даже если поток видит указатель, записанный другим потоком, он может не увидеть только что созданный экземпляр `some_resource`, в результате чего вызывается функция `do_something()` ❹, работающая с неверными значениями. Это пример такого типа гонки, который определяется стандартом C++ как *гонка за данными* и указывается как вызывающая *неопределенное поведение*. Поэтому абсолютно ясно, что подобной гонки следует избегать. Более подробно модель памяти, включая все составляющие гонки за данными, рассматривается в главе 5.

Комитет по стандартизации C++ также понял важность подобного развития событий, и поэтому, чтобы справиться с данной ситуацией, стандартная библиотека

C++ предоставляет компоненты `std::once_flag` и `std::call_once`. Вместо блокировки мьютекса и явной проверки указателя каждый поток может безопасно воспользоваться функцией `std::call_once`, зная, что к моменту возвращения управления из этой функции указатель будет инициализирован каким-либо потоком (синхронизированным образом). Необходимые для этого данные синхронизации хранятся в экземпляре `std::once_flag`, и каждый экземпляр `std::once_flag` соответствует другой инициализации. Задействование функции `std::call_once` обычно связано с меньшими издержками по сравнению с явным использованием мьютекса, особенно когда инициализация уже была выполнена. Поэтому, если она соответствует функциональным требованиям, предпочтение следует отдавать именно ей. Далее показана операция из листинга 3.11, переделанная для использования `std::call_once`. В данном случае инициализация выполнена путем вызова функции, но это легко можно сделать и с помощью экземпляра класса с оператором вызова функции. Подобно большинству функций, имеющихся в стандартной библиотеке, принимающих в качестве аргумента функции или предикаты, функция `std::call_once` работает с любой функцией или объектом, допускающим вызов:

```
std::shared_ptr<some_resource> resource_ptr;
std::once_flag resource_flag;
void init_resource()
{
    resource_ptr.reset(new some_resource);
}
void foo()
{
    std::call_once(resource_flag, init_resource);
    resource_ptr->do_something();
}
```

Инициализация вызывается
только один раз

В этом примере как `std::once_flag` ❶, так и инициализируемые данные являются объектами в области видимости имен, но, как показано в листинге 3.12, функцию `std::call_once()` легко можно использовать для отложенной инициализации компонентов класса.

Листинг 3.12. Потокобезопасная отложенная инициализация компонентов класса с помощью `std::call_once`

```
class X
{
private:
    connection_info connection_details;
    connection_handle connection;
    std::once_flag connection_init_flag;
    void open_connection()
    {
        connection=connection_manager.open(connection_details);
    }
public:
    X(connection_info const& connection_details_):
        connection_details(connection_details_)
    {}
    void send_data(data packet const& data) ❶
```

```

{
    std::call_once(connection_init_flag,&X::open_connection,this);
    connection.send_data(data)
}
data_packet receive_data() ← ❶
{
    std::call_once(connection_init_flag,&X::open_connection,this);
    return connection.receive_data();
}
};

```

В данном примере инициализация выполняется либо при первом вызове `send_data()` ❶, либо при первом вызове `receive_data()` ❸. Использование компонентной функции `open_connection()` для инициализации данных требует также передачи указателя `this`. Как и в других функциях стандартной библиотеки, принимающих вызываемые объекты, такие как конструкторы для `std::thread` и `std::bind()`, это делается передачей `std::call_once()` дополнительного аргумента ❷.

Стоит отметить, что по аналогии с `std::mutex` экземпляры `std::once_flag` нельзя скопировать или переместить, поэтому, если использовать их также в качестве компонента класса, то при необходимости придется определять соответствующие специальные компонентные функции.

Один из сценариев, предполагающих вероятность состояния гонки при инициализации, связан с применением локальной переменной, объявленной с ключевым словом `static`. Инициализация такой переменной определена так, чтобы она выполнялась при первом прохождении потока управления через ее объявление. Это означает, что несколько потоков, вызывающих функцию, в стремлении первыми выполнить определение могут вызвать состояние гонки. На многих компиляторах, предшествующих C++11, состояние гонки создавало реальные проблемы, поскольку на первенство в попытке инициализации переменной могли претендовать сразу несколько потоков или же они могли пытаться использовать ее после начала инициализации, запущенной в другом потоке еще до ее завершения. В C++11 эта проблема была решена: инициализация определена так, чтобы выполняться только в одном потоке, и никакие другие потоки не будут продолжать выполнение до тех пор, пока эта инициализация не будет завершена. Поэтому состояние гонки возникает только вокруг того, какой из потоков выполнит инициализацию, не вызывая более серьезных проблем. Когда нужна только одна глобальная переменная, этим свойством можно воспользоваться в качестве альтернативы `std::call_once`:

```

class my_class;
my_class& get_my_class_instance()
{
    static my_class instance;
    return instance;
}

```

Затем сразу несколько потоков могут безопасно вызвать функцию `get_my_class_instance()` ❶, не опасаясь возникновения гонки при инициализации.

Защита данных только для инициализации является частным случаем более общего сценария — структуры данных с редко проводимыми обновлениями. Прак-

тически всегда эта структура данных предназначена только для чтения и допускает при конкурентности чтение сразу несколькими потоками, но изредка она может нуждаться в обновлениях. Здесь нужен защитный механизм, учитывающий данные обстоятельства.

3.3.2. Защита редко обновляемых структур данных

Рассмотрим таблицу, используемую для хранения кэша DNS-записей, с помощью которых выполняется преобразование доменных имен в соответствующие IP-адреса. Во многих случаях заданная DNS-запись не будет изменяться на протяжении длительного периода времени (DNS-записи остаются неизменными в течение многих лет). Хотя временами, по мере того как пользователи станут обращаться к другим веб-сайтам, в таблице могут появляться новые записи и эти данные могут оставаться практически неизменными в течение всего периода своего существования. Важно периодически проверять правильность кэшированных записей, но обновление по-прежнему понадобится только при изменении каких-либо деталей.

Хотя обновления происходят крайне редко, это все же случается, и если эта кэш-память предназначена для доступа сразу нескольких потоков, в ходе обновления понадобится соответствующая защита, чтобы гарантировать, что ни один из потоков, читающих кэш, не увидит нарушенную структуру данных.

В отсутствие структуры данных, точно соответствующей тому, как она станет использоваться, и специально разработанной для конкурентных обновлений и чтений (таких как в главах 6 и 7), это обновление требует, чтобы выполняющий его поток имел монопольный доступ к структуре данных до завершения операции. После того как обновление структуры данных будет завершено, она снова станет безопасной для конкурентного доступа со стороны нескольких потоков. Поэтому применение `std::mutex` для защиты структуры данных имеет слишком мрачные перспективы, поскольку при этом исключается возможность реализовать конкурентность при чтении структуры данных в тот период, когда она не подвергается модификации, так что нужен другой вид мьютекса. Этот новый тип мьютекса обычно называют мьютексом *чтения* — *записи*, поскольку он допускает два различных типа использования: монопольный доступ для одного потока записи или общий и конкурентный доступ для нескольких потоков чтения.

Стандартная библиотека C++17 предоставляет два полностью готовых мьютекса такого вида, `std::shared_mutex` и `std::shared_timed_mutex`. C++14 предоставляет лишь `std::shared_timed_mutex`, а C++11 не предоставляет ни один из них. Если по каким-то причинам вы вынуждены применять компилятор версии, предшествующей C++14, можно воспользоваться основанной на исходном предложении реализацией, предоставленной библиотекой Boost. Разница между `std::shared_mutex` и `std::shared_timed_mutex` состоит в том, что `std::shared_timed_mutex` поддерживает дополнительные операции (рассматриваются в разделе 4.3), поэтому `std::shared_mutex` на некоторых платформах может обеспечить выигрыш в производительности, если эти дополнительные операции не нужны.

В главе 8 будет показано, что использование такого мьютекса не является панацеей от всех бед и производительность зависит от количества задействованных

процессоров и относительной рабочей нагрузки на читающие и обновляющие потоки. Поэтому большое значение приобретает профилирование производительности кода на целевой системе, позволяющее убедиться, что от дополнительной сложности будут какие-то преимущества.

Для синхронизации вместо экземпляра `std::mutex` используется экземпляр `std::shared_mutex`. Для операций обновления вместо соответствующих специализаций `std::mutex` можно применять `std::lock_guard<std::shared_mutex>` и `std::unique_lock<std::shared_mutex>`. Они обеспечивают монополярный доступ, как и при использовании `std::mutex`. В потоках, которым не нужно обновлять структуру данных, для получения *совместного* доступа вместо этого можно воспользоваться `std::shared_lock<std::shared_mutex>`. Этот шаблон класса RAII был добавлен в C++14 и применяется так же, как и `std::unique_lock`, за исключением того, что несколько потоков могут одновременно получить общую блокировку на один и тот же мьютекс `std::shared_mutex`. Единственное ограничение заключается в том, что, если какой-либо имеющий общую блокировку поток попытается получить монополярную блокировку, он будет блокироваться до тех пор, пока все другие потоки не снимут свои блокировки. Аналогично, если какой-либо поток имеет монополярную блокировку, никакой другой поток не может получить общую или монополярную блокировку, пока не снимет свою блокировку первый поток.

В листинге 3.13 показана реализация простого кэша DNS, похожего на один из рассмотренных, с использованием `std::map` для хранения кэшированных данных, защищенных с помощью `std::shared_mutex`.

Листинг 3.13. Защита структуры данных с помощью `std::shared_mutex`

```
#include <map>
#include <string>
#include <mutex>
#include <shared_mutex>
class dns_entry;
class dns_cache
{
    std::map<std::string, dns_entry> entries;
    mutable std::shared_mutex entry_mutex;
public:
    dns_entry find_entry(std::string const& domain) const
    {
        std::shared_lock<std::shared_mutex> lk(entry_mutex); ← ❶
        std::map<std::string, dns_entry>::const_iterator const it=
            entries.find(domain);
        return (it==entries.end())?dns_entry():it->second;
    }
    void update_or_add_entry(std::string const& domain,
                           dns_entry const& dns_details)
    {
        std::lock_guard<std::shared_mutex> lk(entry_mutex); ← ❷
        entries[domain]=dns_details;
    }
};
```

Имеющаяся в листинге 3.13 функция поиска записи `find_entry()` использует в целях защиты совместного доступа только для чтения экземпляр `std::shared_lock<>` ❶, поэтому сразу несколько потоков могут одновременно и без проблем вызывать `find_entry()`. В то же время в функции обновления или добавления записи `update_or_add_entry()` для предоставления монопольного доступа при обновлении таблицы используется экземпляр `std::lock_guard<>` ❷. В вызове `update_or_add_entry()` не только предотвращается обновление от других потоков, но и блокируются даже те потоки, которые вызывают функцию `find_entry()`.

3.3.3. Рекурсивная блокировка

Попытка потока заблокировать мьютекс, которым он уже владеет, приводит при использовании `std::mutex` к ошибке и *неопределенному поведению*. Но порой бывает нужно, чтобы поток многократно получал один и тот же мьютекс, не разблокируя его предварительно. Для этой цели в стандартной библиотеке C++ предусмотрен класс `std::recursive_mutex`. Он работает так же, как и `std::mutex`, за тем лишь исключением, что на один его экземпляр можно из одного и того же потока получить несколько блокировок. Прежде чем мьютекс сможет быть заблокирован другим потоком, нужно будет снять все ранее установленные блокировки, поэтому, если функция `lock()` вызывается три раза, то три раза должна быть вызвана и функция `unlock()`. При правильном применении `std::lock_guard<std::recursive_mutex>` и `std::unique_lock<std::recursive_mutex>` все это будет сделано за вас автоматически.

В большинстве случаев при возникновении желания воспользоваться рекурсивным мьютексом, скорее всего, требуется внести изменения в саму конструкцию. Рекурсивные мьютексы обычно используются там, где класс предназначен для конкурентного доступа из нескольких потоков, поэтому у него имеется мьютекс, защищающий компонентные данные. Каждая открытая компонентная функция блокирует мьютекс, выполняет свою работу, а затем снимает с мьютекса блокировку. Но иногда требуется, чтобы одна открытая компонентная функция вызывала как часть своей работы другую компонентную функцию. В таком случае вторая компонентная функция также попытается заблокировать мьютекс, что приведет к неопределенному поведению. Решить эту задачу, что называется, в лоб можно, заменив мьютекс рекурсивным мьютексом. Это позволит успешно завершить блокировку мьютекса во второй компонентной функции, и она сможет продолжить работу.

Но применять такой вариант не рекомендуется: можно принять легкомысленные решения и разработать неудачную конструкцию. В частности, пока удерживается блокировка, инварианты класса обычно нарушаются, а это означает, что вторая компонентная функция должна работать даже при вызове с нарушенными инвариантами. Лучше выделить новую закрытую компонентную функцию, вызываемую из обеих компонентных функций, которая не блокирует мьютекс (в предположении, что он уже заблокирован). Затем тщательно продумать условия, при которых можно вызвать новую функцию, и то, в каком состоянии будут при этом находиться данные.

Резюме

В этой главе рассматривались роковые последствия проблем, связанных с состоянием гонки в условиях совместного использования данных несколькими потоками, и способы, позволяющие их избежать с помощью `std::mutex` и тщательно продуманной конструкции интерфейса. Было показано, что мьютексы не являются панацеей от всех бед и имеют свои проблемы в виде взаимной блокировки, хотя стандартная библиотека C++ предоставляет инструмент `std::lock()`, позволяющий ее избежать. Затем разговор шел о дополнительных методах предотвращения взаимных блокировок и мы кратко рассмотрели темы, связанные с передачей владения блокировкой, и проблемы, вызываемые выбором соответствующей степени детализации блокировки. И наконец, ознакомились с такими альтернативными средствами защиты данных, как `std::call_once()` и `std::shared_mutex`, применяемыми для решения задач в особых случаях.

Но пока мы не поднимали тему, касающуюся ожидания получения входных данных от других потоков. Если ваш потокобезопасный стек пуст, он выдает исключение, поэтому, если один поток вынужден дожидаться другого, помещающего значение в стек (в чем, собственно, и заключается основное предназначение потокобезопасного стека), то ему придется снова и снова пытаться извлечь значение, повторяя эту попытку после каждой выдачи исключения. Из-за этого ценное процессорное время впустую расходуется на проверку, а не на полезную работу. Более того, постоянная проверка может мешать прогрессу, не разрешая выполняться другим имеющимся в системе потокам. Нужен некий способ, чтобы поток мог ожидать завершения задачи, выполняемой другим потоком, не расходуя на это время центрального процессора. Глава 4, материал которой построен на основе уже рассмотренных средств защиты разделяемых данных, знакомит с различными механизмами синхронизации операций между потоками в C++. А в главе 6 показано, как все это можно применить для создания более крупных, многократно используемых структур данных.